

GPT

Генеративные модели
Елена Кантонистова

ВШЭ

План рассказа

- О чем курс? Что можно генерировать?
- Ресар: трансформер и attention
- GPT для текстов
- Авторегрессионные модели для генерации изображений

Введение

Что можно генерировать?

- **Текст** (статьи, письма, код, сценарии)
- **Изображения** (иллюстрации, фотографии, дизайн)
- **Видео** (ролики, анимации, аватары)
- **Аудио** (речь, музыка, звуковые эффекты)
- **3D-объекты** (модели, сцены, игровые ассеты)
- **Программный код** (функции, приложения, тесты)
- **Структурированные данные** (таблицы, SQL, JSON, XML)
- **Мультимодальный контент** (комбинации текста, изображений, аудио и видео)

Что можно генерировать?



О чем курс?

- Генеративные модели NLP, CV, аудио, мультимодальные модели
- Акцент на генерации изображений

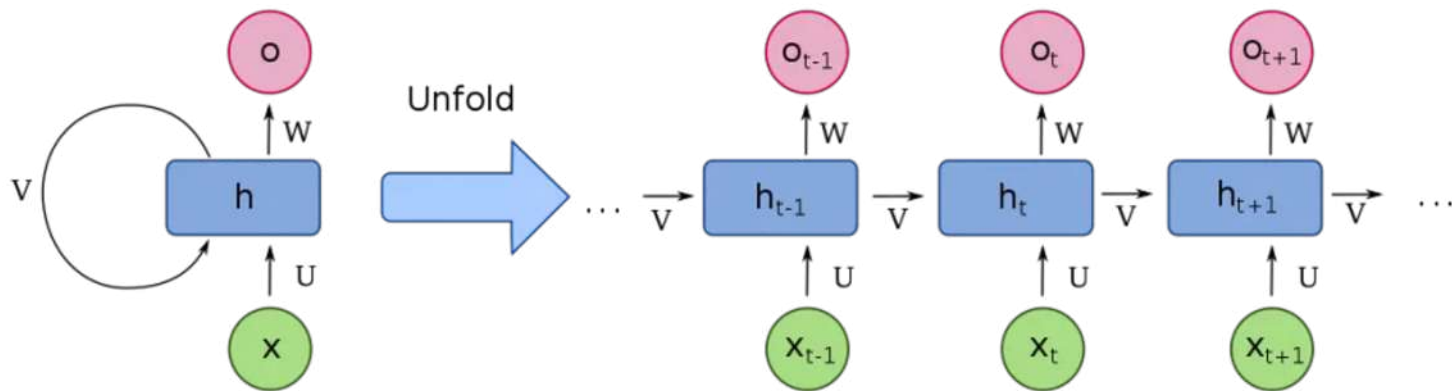
Смещение акцентов с развитием AI

- Более комплексные AI-решения (агенты, мультимодальность, интеграция с внешними системами)
- Смещение акцента с написания кода на постановку задач, управление ИИ и валидацию результатов
- Автоматизация рутинной разработки, больше времени на проектирование и принятие решений
- Рост роли предметной области (Domain Knowledge) — понимание бизнеса становится важнее
- Повышение требований к качеству данных, оценке моделей и надежности решений
- Смещение фокуса с вопроса «Как реализовать?» к вопросам «Что делать?», «Зачем делать?» и «Какую ценность это создаст?»

Ресар: трансформер

Рекуррентные сети (напоминание)

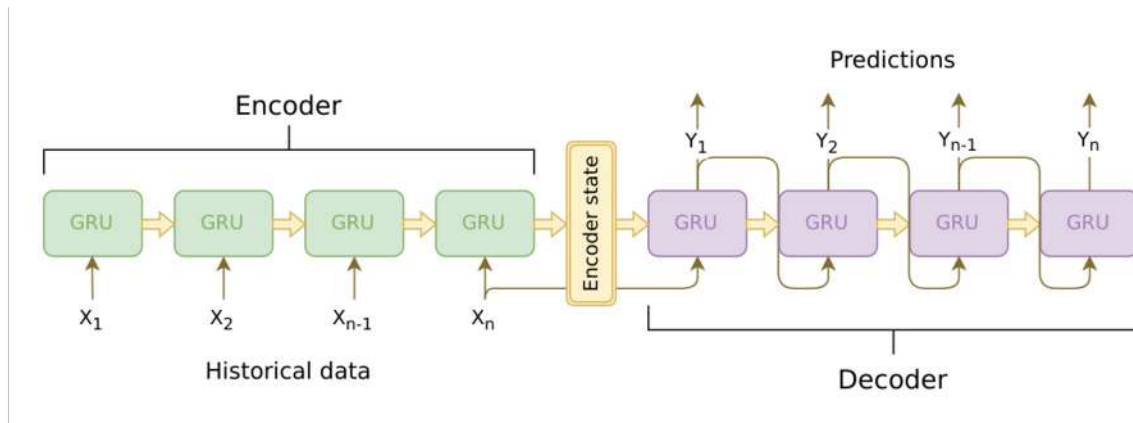
Рекуррентные нейронные сети (RNN) последовательно читают текст и агрегируют в скрытом состоянии h информацию о прочитанном.



Рекуррентные сети в задаче seq2seq

В задаче seq2seq архитектура имеет структуру encoder-decoder:

- Encoder последовательно считывает текст и агрегирует информацию в вектор
- Decoder, получая на вход вектор из encodera и предыдущее свое состояние, последовательно генерирует целевой текст



Механизм внимания

Недостатки RNN:

- Вся информация о тексте в RNN хранится в векторе скрытого состояния. Этот вектор обновляется со временем, и потому информация о словах, которые были в начале текста, быстро забывается (даже в LSTM и GRU)
- Также проблемой является невозможность распараллеливания обучения RNN, то есть для получения вектора скрытого состояния модели на шаге $t+1$ нам нужно вычислить вектор скрытого состояния для шага t и так далее. Вычисления занимают много времени и неэффективно расходуют мощности GPU.

Нужен способ считывать текст таким образом, чтобы в каждый момент времени можно было получить доступ к информации о любом слове (или кусочке текста) из прошлого за фиксированное время. Также эта информация не должна теряться при дальнейшем чтении текста.

Self-attention

Механизм внутреннего внимания (self-attention) используется, чтобы посмотреть на другие слова во входной последовательности во время кодирования конкретного слова. Механизм был впервые описан в статье Attention is all you need в 2017 году.

Пример.

Пусть нейронная сеть получает на вход два предложения

«Мама мыла раму. Она держала в руках тряпку».

Местоимение «она» относится к маме или к раме?

Для человека это очень простой вопрос, но для модели машинного обучения – нет. Self-attention помогает выучить взаимосвязи между токенами в последовательности, моделируя смысл и влияние других слов в тексте при обработке текущего токена.

Self-attention

На первом шаге из входного вектора (например, эмбединга каждого токена) формируются три вектора: Query (запрос), Key (ключ) и Value (значение). Получаются эти вектора путем умножения входного вектора на матрицы W_Q , W_K и W_V , веса которых обучаются вместе со всеми остальными параметрами модели с помощью обратного распространения ошибки.

Each vector receives three representations ("roles")

$$\begin{bmatrix} W_Q \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{blue} \\ \text{blue} \\ \text{blue} \end{bmatrix}$$

Query: vector from which the attention is looking

"Hey there, do you have this information?"

$$\begin{bmatrix} W_K \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{yellow} \\ \text{yellow} \\ \text{yellow} \end{bmatrix}$$

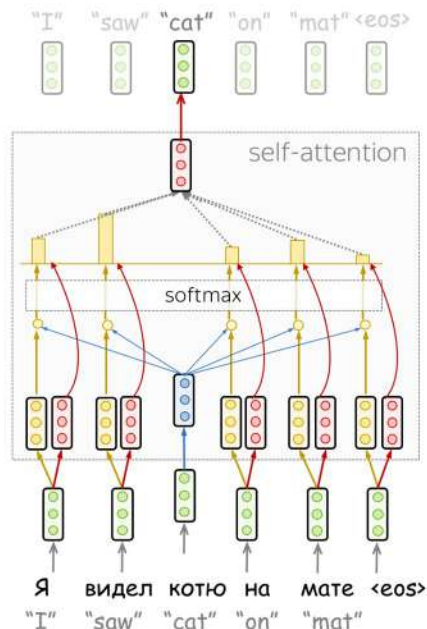
Key: vector at which the query looks to compute weights

"Hi, I have this information – give me a large weight!"

$$\begin{bmatrix} W_V \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{red} \\ \text{red} \\ \text{red} \end{bmatrix}$$

Value: their weighted sum is attention output

"Here's the information I have!"



Зачем нужны три сущности в self-attention?

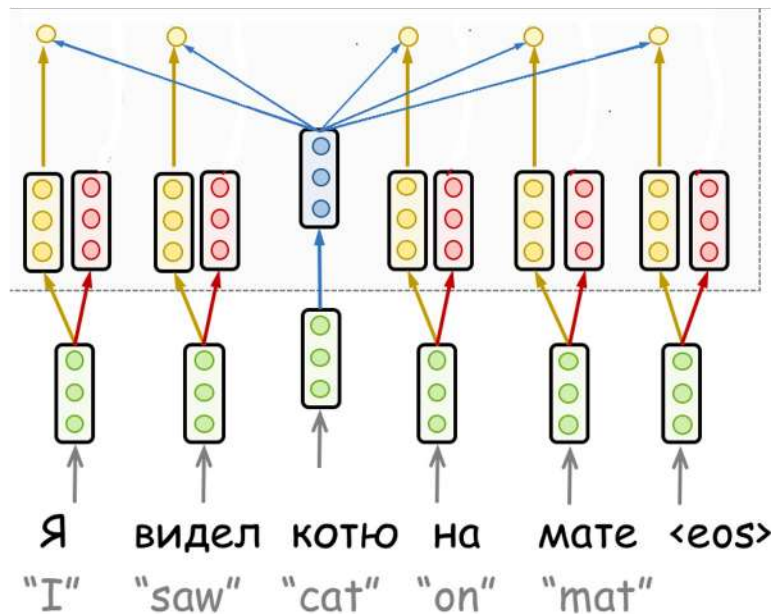
Выделение этих трех сущностей нужно, чтобы разграничить эмбединги, задающие «направление» внимания (query, key) и смысловую часть токена (value).

- *Вектор query задает модальность «начальной точки» механизма внутреннего внимания (от какого токена направлено внимание)*
- *Вектор key – модальность «конечной точки» (к какому токеноу направлено внимание).*

Таким образом, один и тот же токен может выступать как «начальной», так и «конечной» точкой направления внимания: self-attention вычисляется между всеми токенами в выбранном фрагменте текста.

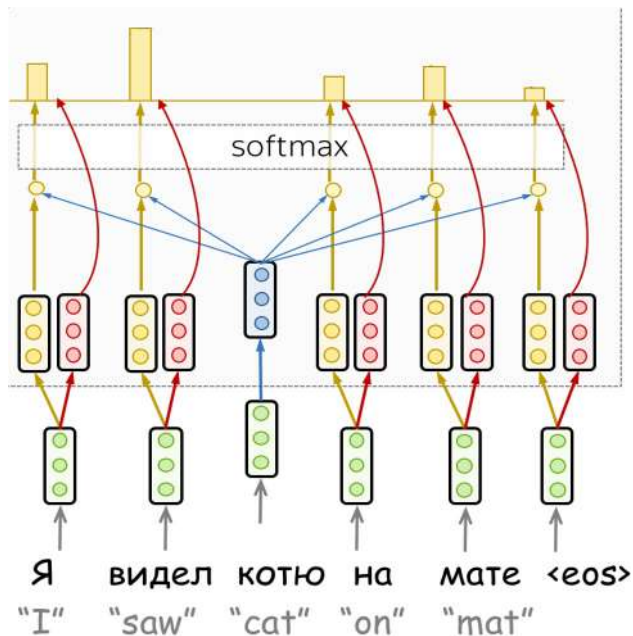
Алгоритм вычисления self-attention

- По очереди фиксируется каждый token (становится **query**) и просчитывается степень его связанности со всеми оставшимися токенами.
- Для этого поочередно **key-векторы всех токенов текста скалярно умножаются на query-вектор текущего токена**. Полученные числа будут показывать, насколько важны остальные токены при кодировании query токена в конкретной позиции.



Алгоритм вычисления self-attention

- Дальше полученный **вектор весов внимания нормализуется** путем применения нормализации softmax (получаем вероятности).

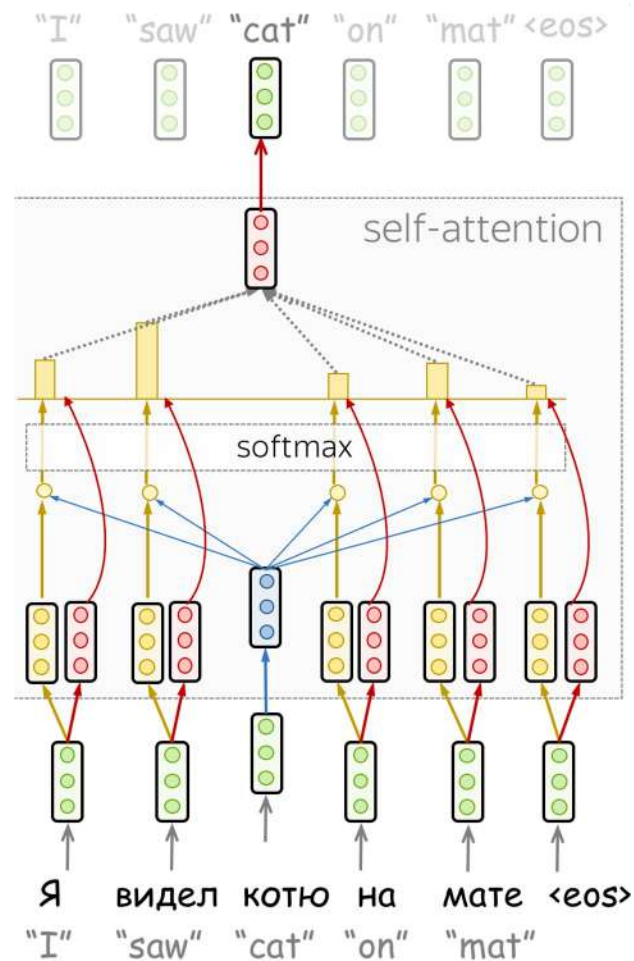


Алгоритм вычисления self-attention

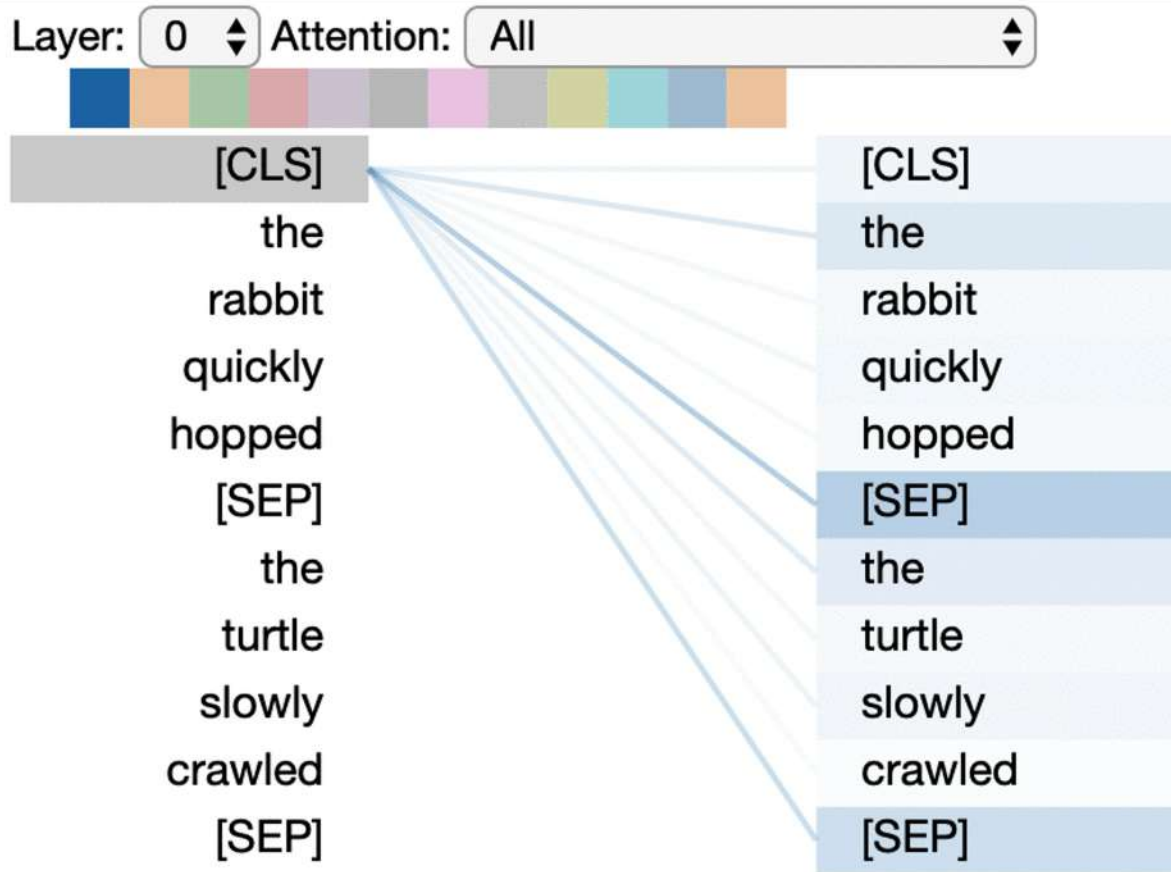
- Наконец, *подсчитывается взвешенная сумма value-векторов, где в качестве весов используются полученные на предыдущем шаге вероятности.*

Полученный вектор и будет выходом слоя self-attention для одного токена.

$$a(q, k, v) = \text{softmax}\left(\frac{qk^T}{\text{norm}}\right)v$$



Self-attention

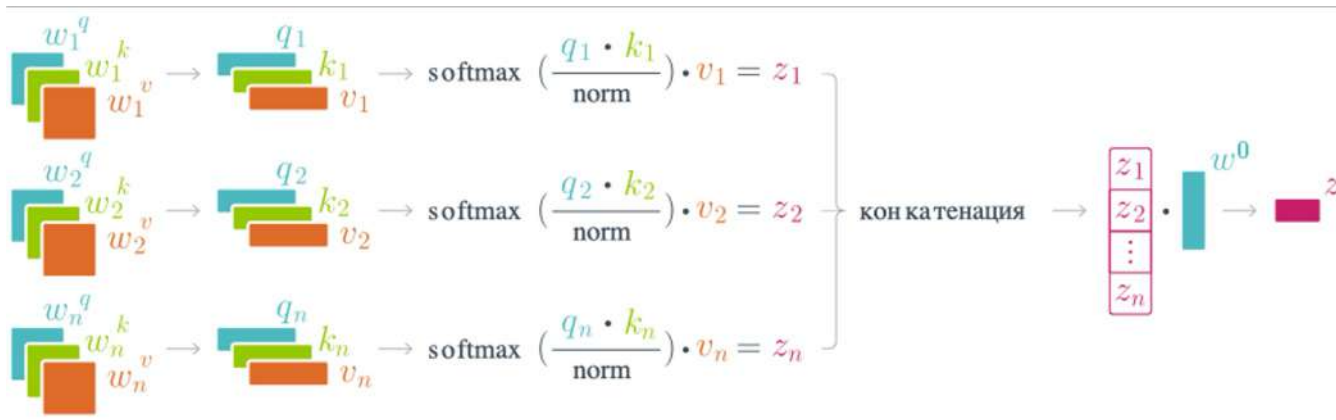


Многоголовое внутреннее внимание

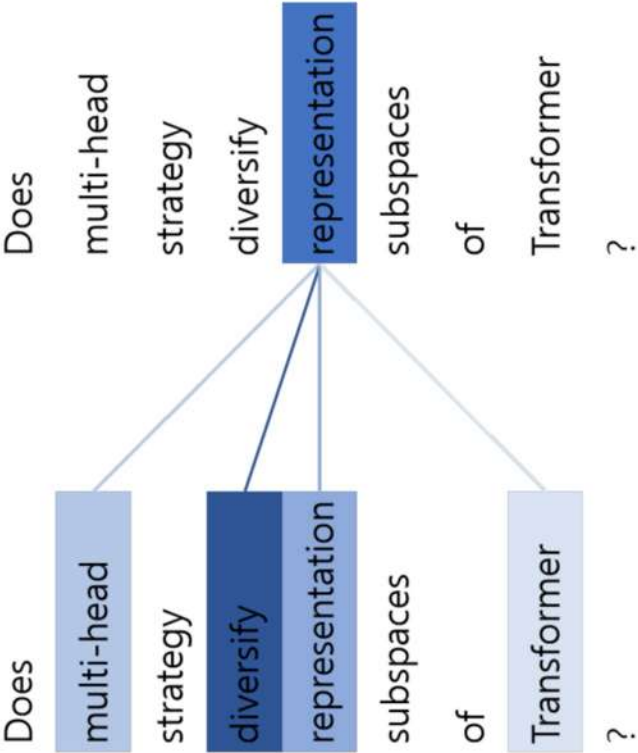
В архитектурах трансформеров обычно используют параллельно несколько self-attention блоков. Такая схема называется ***multi-head self-attention*** или **многоголовым внутренним вниманием**.

- Вычисление self-attention происходит несколько раз с разными матрицами весов
- Затем полученные матрицы конкатенируются и умножаются на еще одну матрицу весов W^O

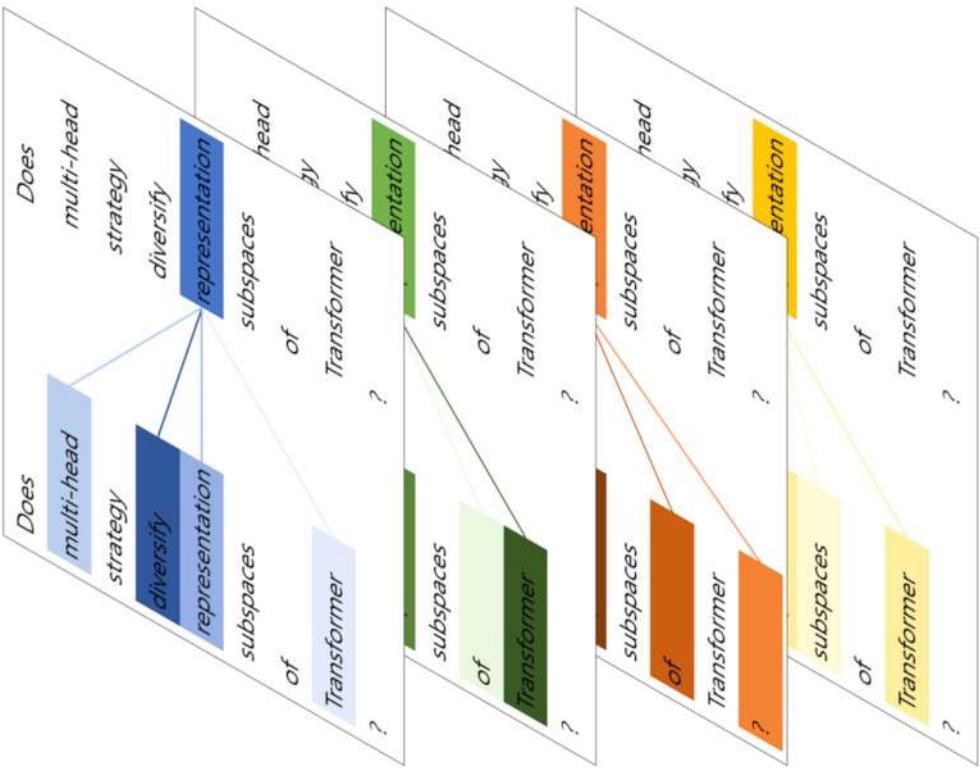
Это позволяет разным self-attention головам фокусироваться на разных взаимосвязях, например, одна голова может отвечать за признаковые описания, другая за действия, третья за отношения «объект-субъект».



Multihead attention



(a) Single-head attention



(b) Multi-head attention

Трансформеры

Трансформеры - это архитектуры нейронных сетей, построенные с использованием механизма внимания (без рекуррентных и без сверточных слоев).

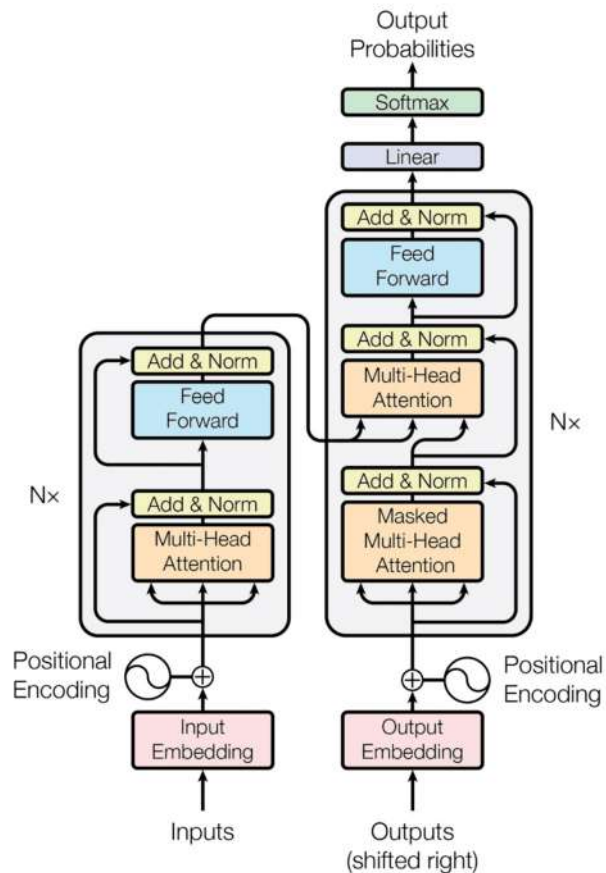
Трансформеры как архитектура появились в 2017 году и стали золотым стандартом в задачах NLP, а сейчас и в других областях (computer vision, deep learning for sound и так далее).

Зачем нам трансформеры?

- *Мы уже знаем, что рекуррентные нейронные сети содержат всю информацию о тексте в одном скрытом состоянии, что неудобно и зачастую малоинформативно для длинных текстов*
- *Кроме того, обучение RNN тяжело распараллелить в силу их идеи (последовательное чтение текста).*

Мы хотим избавиться от рекуррентности во всех частях архитектуры encoder-decoder, а также кодировать тексты с использованием механизма внимания.

Трансформеры



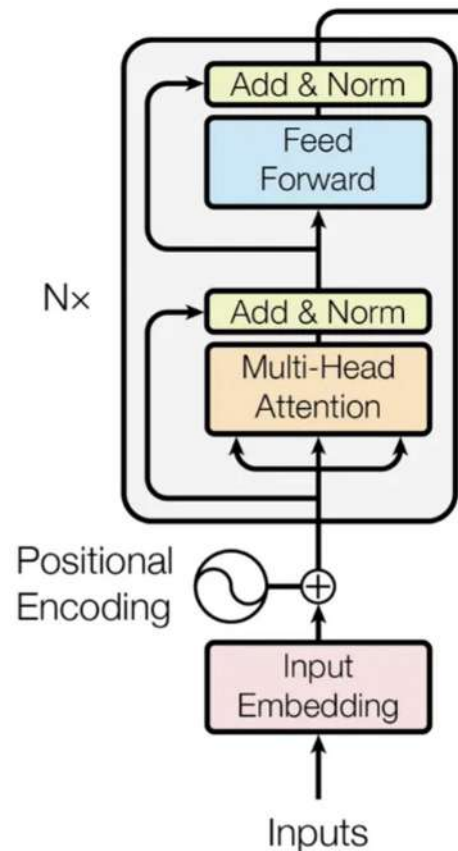
Encoder (кодировщик)

Посмотрим внимательнее на кодировщик (encoder - часть на схеме слева):

Кодировщик состоит из:

- Нескольких слоев self-attention (это Multi-Head Attention на схеме)
- Полносвязного слоя (на схеме это Feed Forward)
- Слоя Layer Normalization, и residual connection (на схеме это Add & Norm)

При этом каждый блок выдаёт последовательность такой же длины, как и исходная.

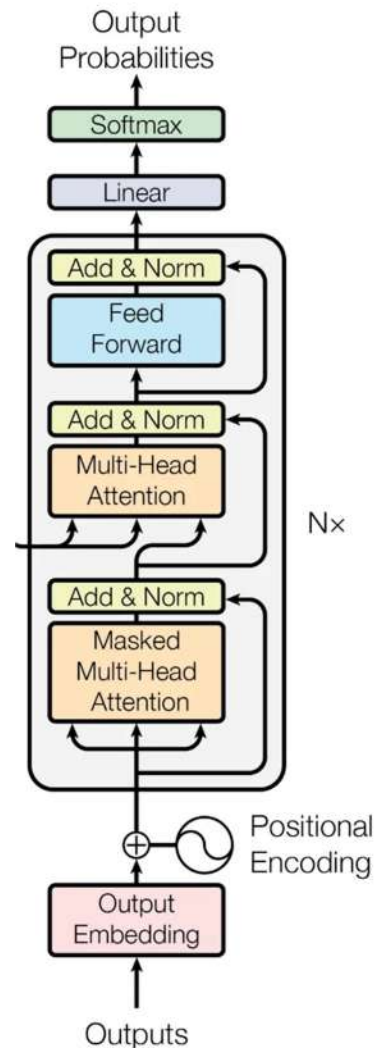


Decoder (декодировщик)

Декодировщик состоит из:

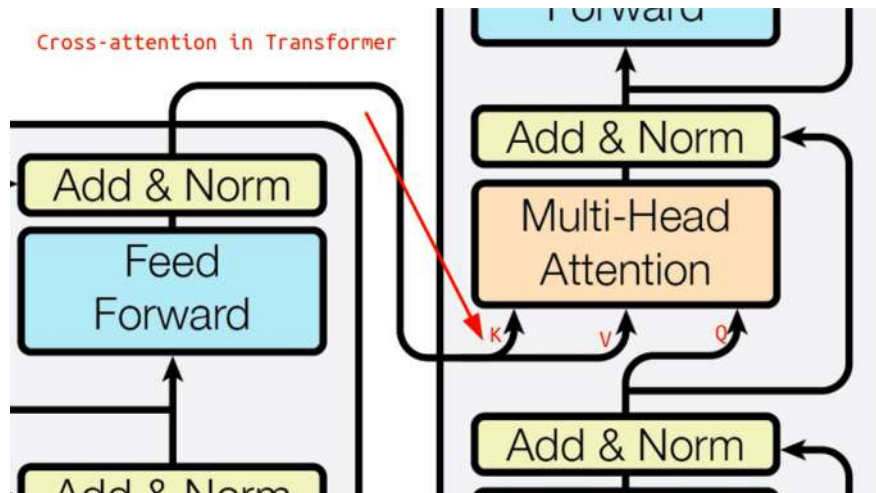
- блока из нескольких self-attention слоев (на схеме это Masked Multi-Head Attention)
- блока из нескольких attention (или cross-attention) слоев (на схеме это Multi-Head Attention)
- полносвязного слоя (на схеме это Feed Forward)
- слоя Layer Normalization, и residual connection (на схеме это Add & Norm)

При этом каждый блок выдаёт последовательность такой же длины, как и исходная.



Decoder (декодировщик): cross-attention

В этом блоке декодировщик получает информацию из кодировщика. Для этого запросы (query) берутся из выходной последовательности, а ключи (key) и значения (value) — из входной (то есть из результатов работы кодировщика).

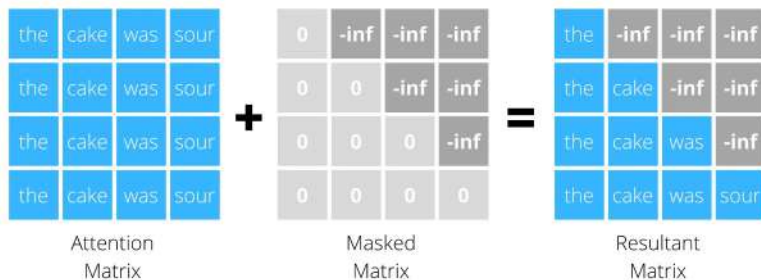


Decoder (декодировщик): masked attention

На этапе генерации текста декодировщиком мы не можем смотреть на весь выходной текст (так как следующие за текущим токены еще не сгенерированы), мы генерируем по одному токenu за шаг, и доступ к последующим шагам на этапе обучения приведёт к утечке информации в кодировщике, а значит, к переобучению модели.

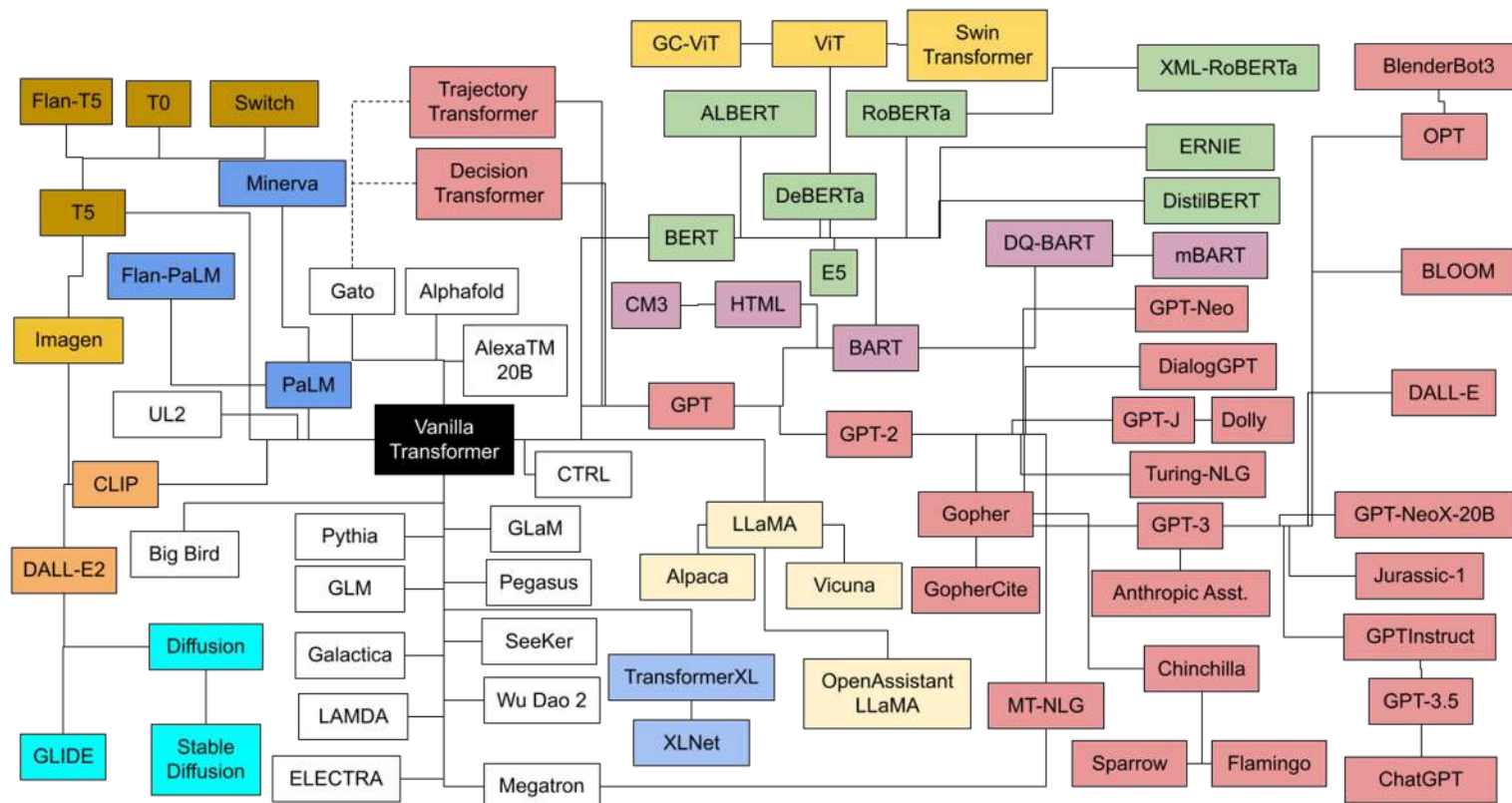
Чтобы избежать этой проблемы, при обучении к вниманию нужно применять *маску*, вручную обращая в $-\infty$ веса до применения нормализации softmax для токенов из будущего, чтобы после softmax их вероятности стали нулевыми.

Masked Attention



*instead of words there will be attention weight

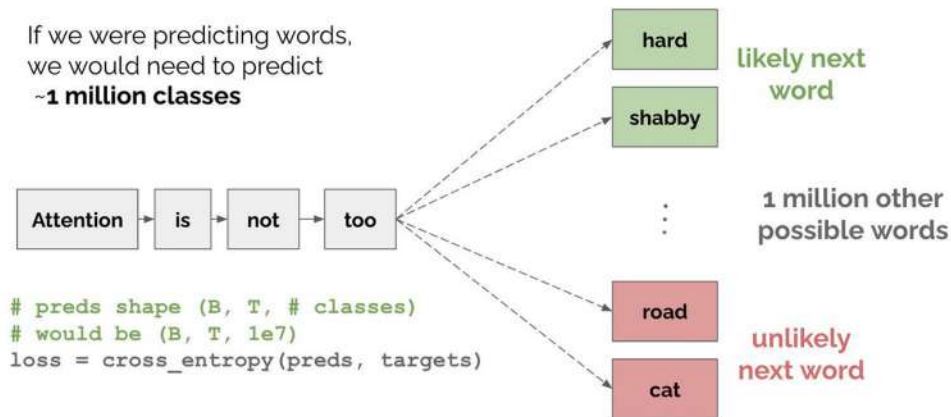
Зоопарк моделей



GPT

GPT (Generative Pretrained Transformer)

- Модель **GPT** хронологически появилась раньше, чем BERT (кодировщик) и представляет собой обычную языковую модель, реализованную в виде последовательности слоев Transformer-декодера.
- В качестве задачи при обучении выступает обычное предсказание следующего токена (то есть многоклассовая классификация по словарю).



GPT Loss

$$\mathcal{L}_{CE} = - \sum_{i=1}^V y_i \log \hat{y}_i$$

- V — размер словаря;
- y_i — one-hot-вектор правильного токена;
- $\hat{y}_i$ — вероятность, предсказанная моделью.

GPT Loss

1

Кросс-энтропия (общая форма)

$$L_{CE} = - \sum_{i=1}^V y_i \log \hat{y}_i$$

V — размер словаря

y_i — истинное распределение

$\hat{y}_i$ — вероятность, предсказанная моделью



2

Истинное распределение — one-hot

$$y = [0, 0, \dots, \underbrace{1}_{\text{правильный токен}}, \dots, 0]$$

правильный токен = k



$$L_{CE} = - \log \hat{y}_k$$

Все $y_i = 0$, кроме $y_k = 1$.

Остаётся только член
с правильным токеном k .

GPT Loss

2

Истинное распределение — one-hot

$$y = [0, 0, \dots, \underset{\text{правильный токен} = k}{1}, \dots, 0]$$

правильный токен = k

$$L_{\text{CE}} = -\log \hat{y}_k$$

Все $y_i = 0$, кроме $y_k = 1$.
Остаётся только член
с правильным токеном k .

3

Интерпретация для GPT

$$\hat{y}_k = P(x_t \mid x_{<t})$$

$\hat{y}_k$ — это вероятность, которую модель
присвоила правильному следующему токenu x_t
при условии предыдущего контекста $x_{<t}$.

4

Loss для одного токена

$$L_t = -\log P(x_t \mid x_{<t})$$

Это отрицательный логарифм вероятности
правильного следующего токена x_t
на позиции t .

GPT Loss

4

Loss для одного токена

$$L_t = -\log P(x_t \mid x_{<t})$$

Это отрицательный логарифм вероятности правильного следующего токена x_t на позиции t .



5

Loss для всей последовательности (GPT loss)

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log P(x_t \mid x_{<t})$$

Усредняем (или суммируем) по всем позициям t в последовательности длины T .

GPT: Обучение

- GPT учится **языковому моделированию**
- Благодаря способности Трансформера масштабироваться, GPT запоминает очень много информации в процессе обучения
- Любую задачу NLP можно свести к **генерации текста**
Классификация – генерация индекса класса
- Из-за этого GPT отлично дообучается на частные задачи

	Параметры	Данные
GPT-1	117M	5 гб
GPT-2	1.5B	40 гб
GPT-3	175B	45 тб

Zero-shot и Few-shot для GPT

- Модель училась только предсказывать следующее слово, однако ее можно применять для новых задач
- Качество такого подхода не очень высокое.

Zero-shot

режим:

Переведи с русского на английский:

дом =>

Few-shot

режим

Переведи с русского на английский:

стол => table

сыр => cheese

дом =>

Бенчмарки для LLM

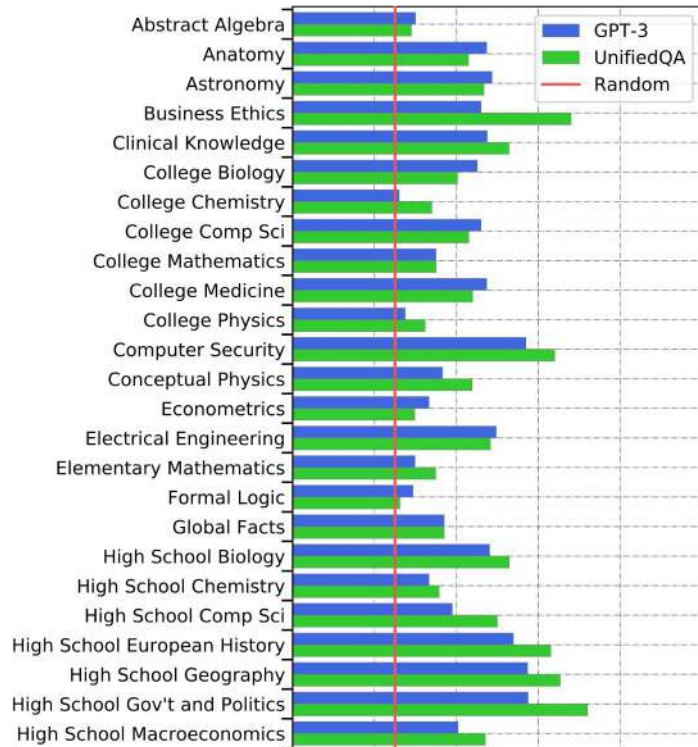
MMLU (Massive Multitask Language Understanding)

Вопросы из разных областей с 4-мя вариантами ответа

Question: John Doe is a software engineer working for a company that develops autonomous weapons. He has serious ethical concerns about the potential misuse of this technology, which could lead to civilian casualties. However, he also has a family to support and fears losing his job if he speaks out. What is the primary ethical conflict John is facing?

- A) Conflict of interest*
- B) Whistleblower's dilemma*
- C) Professional negligence*
- D) Intellectual property rights*

Answer: B



Бенчмарки для LLM

- Коллекция датасетов, на которых можно "легко" проверить ответ
- Обычно это ответы на вопросы

Варианты тестирования:

- Вероятность правильного ответа
- Доля правильных ответов

Scenario	Task	What	Who
NarrativeQA	short-answer question answering	passages are books and movie scripts, questions are unknown	annotators from summaries
NaturalQuestions (closed-book)	short-answer question answering	passages from Wikipedia, questions from search queries	web users
NaturalQuestions (open-book)	short-answer question answering	passages from Wikipedia, questions from search queries	web users
OpenbookQA	multiple-choice question answering	elementary science	Amazon Mechanical Turk workers
MMLU	multiple-choice question answering	math, science, history, etc.	various online sources

Данные

- **Идея:** использовать весь чистый интернет
- **Проблема:** интернет — грязный и не отражает того, что нам нужно.

Данные

- **Идея:** использовать весь чистый интернет
- **Проблема:** интернет — грязный и не отражает того, что нам нужно.
- **Как на самом деле:**
 1. Скачать весь интернет. Common Crawl: 250 миллиардов страниц, (> 1 ПБ)

Данные

- **Идея:** использовать весь чистый интернет
- **Проблема:** интернет — грязный и не отражает того, что нам нужно.
- **Как на самом деле:**
 1. Скачать весь интернет. Common Crawl: 250 миллиардов страниц, (> 1 ПБ)

[illegible]

Данные

- **Идея:** использовать весь чистый интернет
- **Проблема:** интернет — грязный и не отражает того, что нам нужно.
- **Как на самом деле:**
 1. Скачать весь интернет. Common Crawl: 250 миллиардов страниц, (> 1 ПБ)
 2. Извлечение текста из HTML (сложности: математика, шаблоны)

Данные

- **Идея:** использовать весь чистый интернет
- **Проблема:** интернет — грязный и не отражает того, что нам нужно.
- **Как на самом деле:**
 1. Скачать весь интернет. Common Crawl: 250 миллиардов страниц, (> 1 ПБ)
 2. Извлечение текста из HTML (сложности: математика, шаблоны)
 3. Фильтрация нежелательного контента

Например, NSFW, вредоносного контента, персональных данных

Данные

- **Идея:** использовать весь чистый интернет
- **Проблема:** интернет — грязный и не отражает того, что нам нужно.
- **Как на самом деле:**
 1. Скачать весь интернет. Common Crawl: 250 миллиардов страниц, (> 1 ПБ)
 2. Извлечение текста из HTML (сложности: математика, шаблоны)
 3. Фильтрация нежелательного контента
Например, NSFW, вредоносного контента, персональных данных
 4. Дедупликация (url/документ/строка).
Например, все заголовки/нижние колонтитулы/меню на форумах всегда одинаковы.

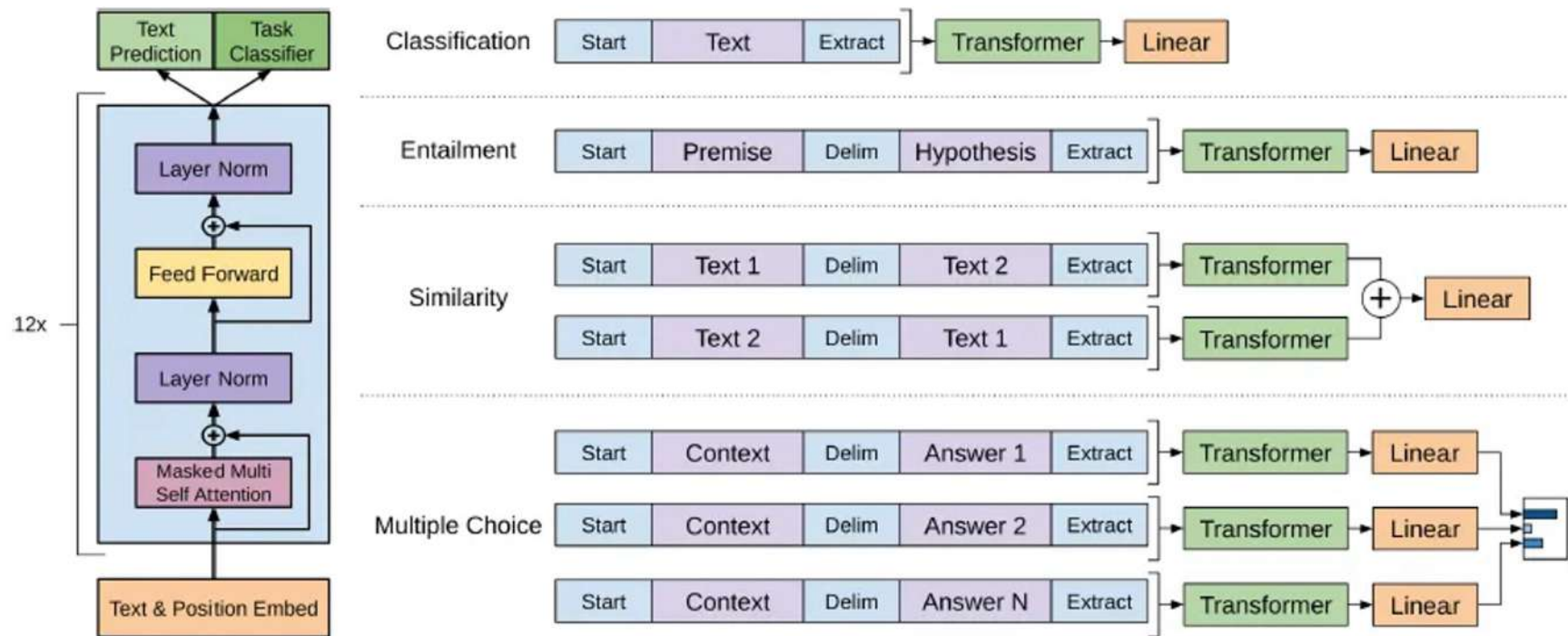
Данные

- **Идея:** использовать весь чистый интернет
- **Проблема:** интернет — грязный и не отражает того, что нам нужно.
- **Как на самом деле:**
 1. Скачать весь интернет. Common Crawl: 250 миллиардов страниц, (> 1 ПБ)
 2. Извлечение текста из HTML (сложности: математика, шаблоны)
 3. Фильтрация нежелательного контента
Например, NSFW, вредоносного контента, персональных данных
 4. Дедупликация (url/документ/строка).
Например, все заголовки/нижние колонтитулы/меню на форумах всегда одинаковы.
 5. Эвристическая фильтрация. Отсеивание низкокачественных документов. Например, количество слов, длина слова, выбросы, грязные фрагменты

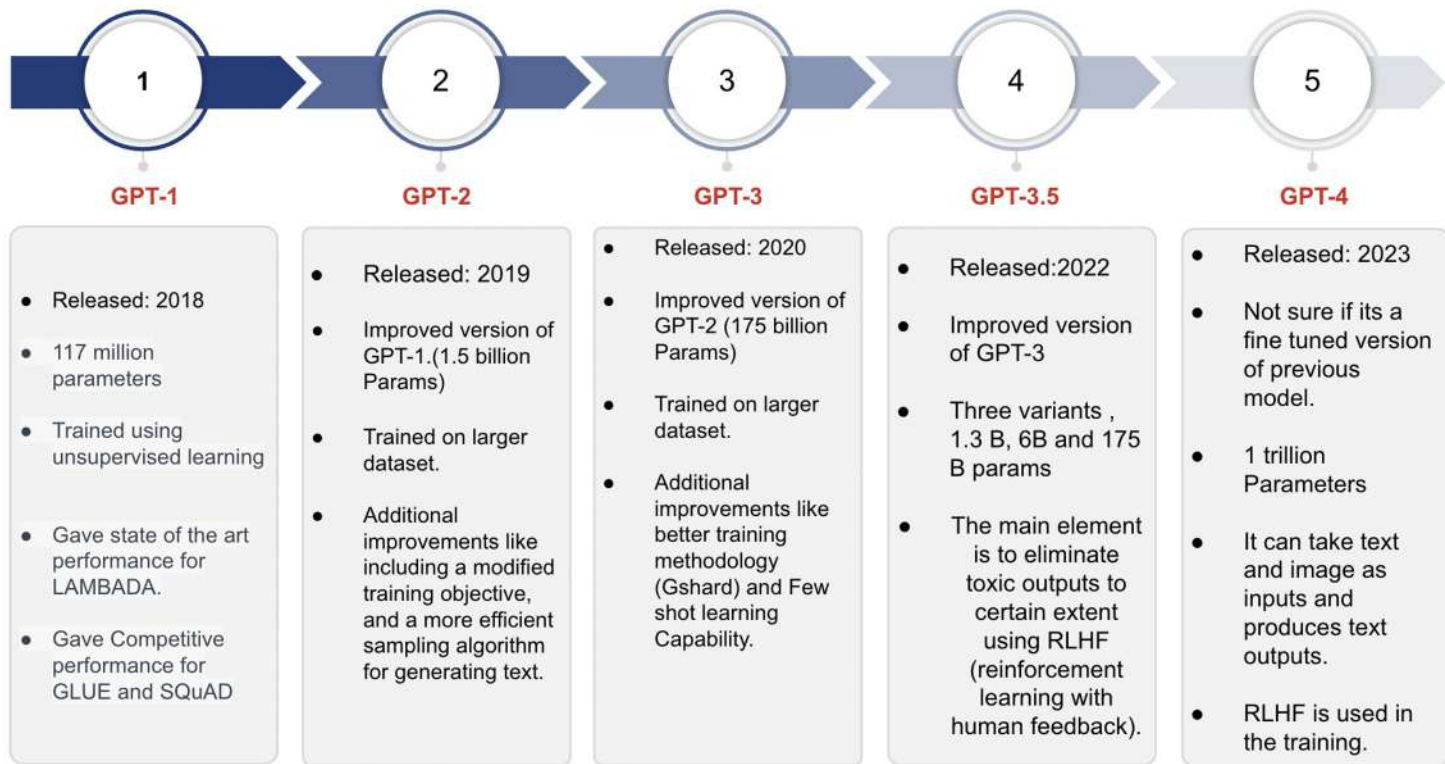
Данные

- **Идея:** использовать весь чистый интернет
- **Проблема:** интернет — грязный и не отражает того, что нам нужно.
- **Как на самом деле:**
 1. Скачать весь интернет. Common Crawl: 250 миллиардов страниц, (> 1 ПБ)
 2. Извлечение текста из HTML (сложности: математика, шаблоны)
 3. Фильтрация нежелательного контента
Например, NSFW, вредоносного контента, персональных данных
 4. Дедупликация (url/документ/строка).
Например, все заголовки/нижние колонтитулы/меню на форумах всегда одинаковы.
 5. Эвристическая фильтрация. Отсеивание низкокачественных документов. Например, количество слов, длина слова, выбросы, грязные фрагменты
 6. Фильтрация на основе модели. Предсказываем, может ли страница быть ссылкой из Википедии.

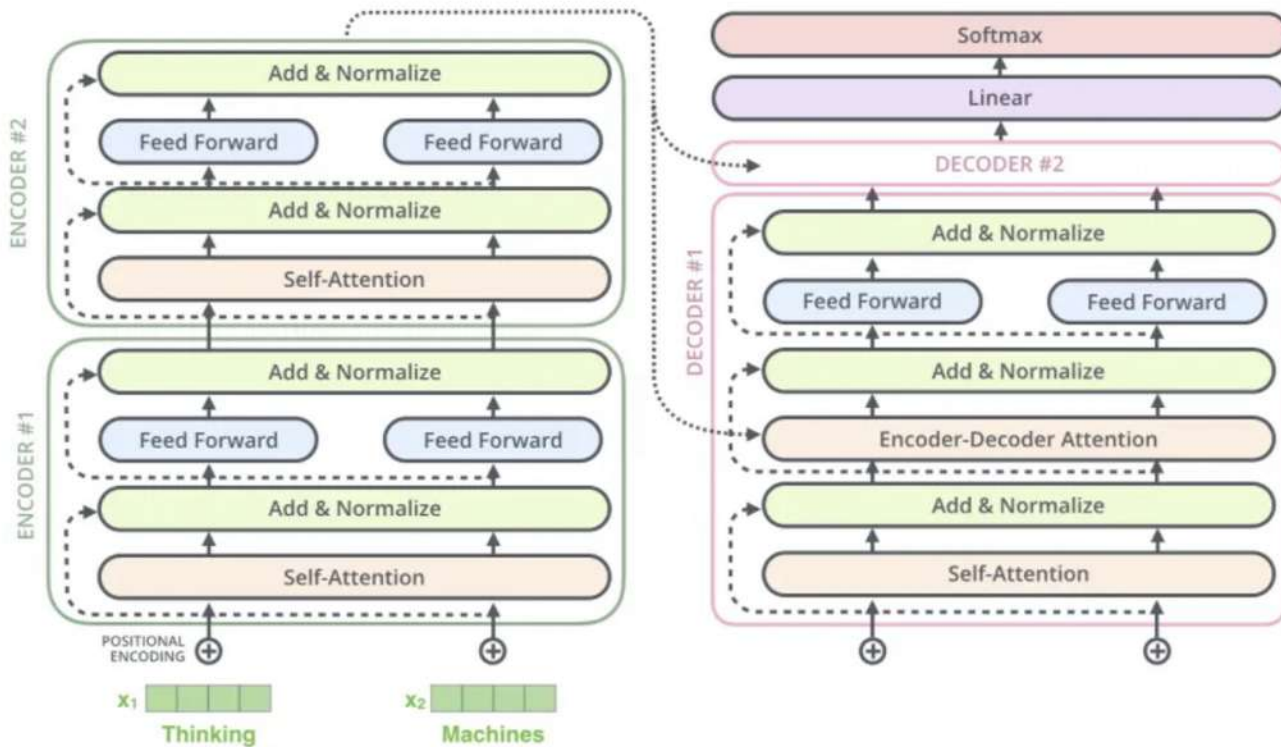
Архитектура и задачи GPT-1



Сравнение GPT-моделей



T5 (Text-to-Text Transformer)



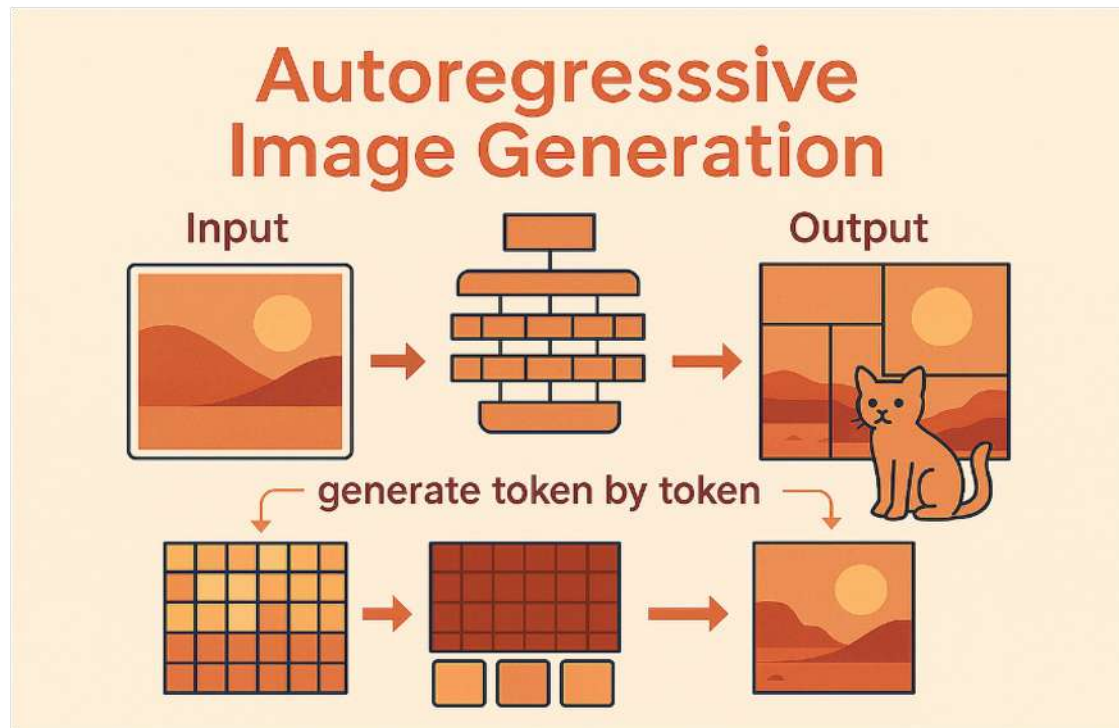
T5 (Text-to-Text Transformer) - почему больше не нужен

- Первый Transformer был архитектурой Encoder–Decoder для задачи Text-to-Text, унаследовав саму идею Encoder–Decoder от RNN Seq2Seq.
- В дальнейшем появились специализированные архитектуры (BERT, GPT и др.), которые покрывают абсолютное большинство текущих задач.

Авторегрессионные модели в компьютерном зрении

Авторегрессионные модели в Computer Vision

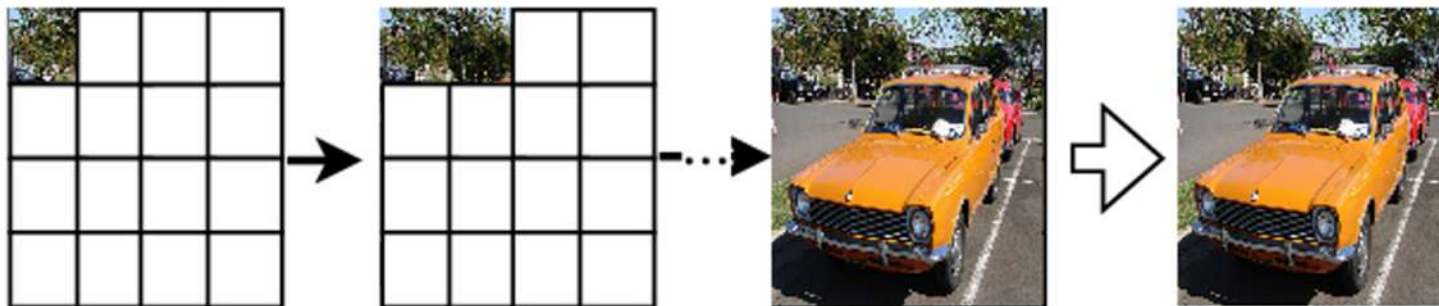
Авторегрессионные CV-модели учатся генерировать изображения “токен за токеном”.



Авторегрессионные модели в Computer Vision

- Токен картинки (или же patch) – это кусочек изображения, одна “клеточка”.
- Изображение разбивается на патчи, затем эти патчи векторизуются (для каждого патча модель выучит числовой вектор-embedding) и подаются в модель.

Next-Patch Prediction [Taming Transformers, 2020]



Как работает авторегрессионная генерация изображений

- Изображение рассматривается как последовательность: пиксели или патчи выстраиваются в растровом порядке — слева направо, сверху вниз
- Совместное распределение раскладывается в цепочку условных: $p(x) = p(x_1) \cdot p(x_2|x_1) \cdot \dots \cdot p(x_n|x_1, \dots, x_{n-1})$
- На каждом шаге модель предсказывает следующий элемент по всем предыдущим — ровно так же, как GPT предсказывает следующий токен текста
- Masked self-attention запрещает «подглядывать в будущее», обучение — максимизация правдоподобия (кросс-энтропия по словарю значений)
- Классика подхода: PixelRNN и PixelCNN (2016), Image Transformer (2018)

Спасибо за внимание!